Software Engineering

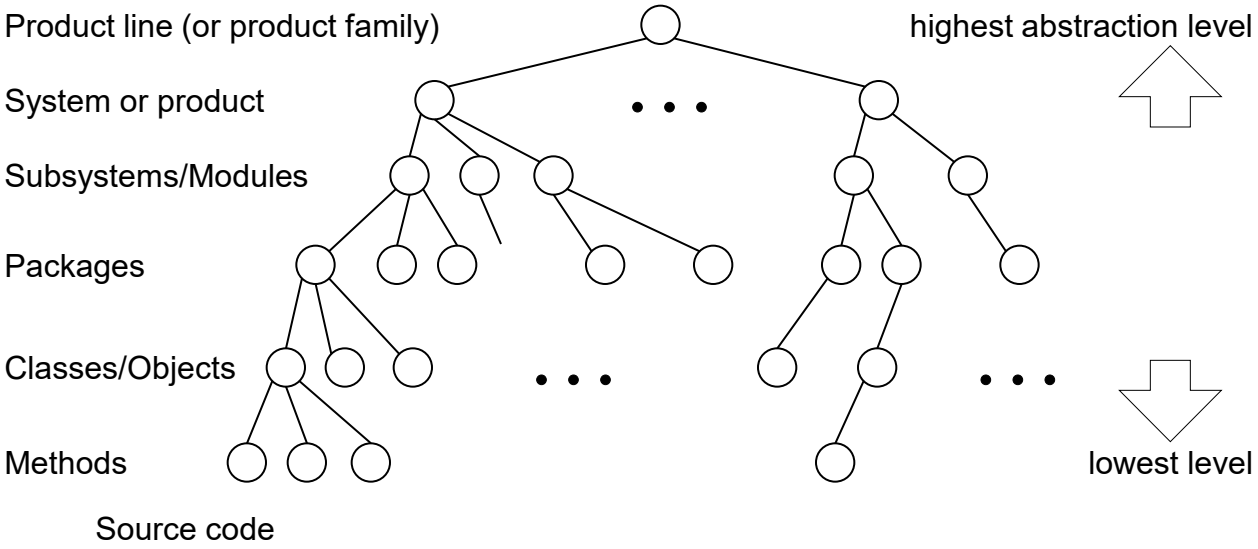
Software Architecture

Homework : Reading from page 80 to 81

Topics

- Software Architecture Definition
- Architectural Decisions & Key Concerns
- Architecture Styles
- Documenting Architecture: Views
- Problem Structure vs. Solution Structure

Hierarchical Organization of Software



- Software is not one long list of program statements but it has **structure**
- The above is a **taxonomy** of structural parts (abstraction hierarchy), but not representation of relationships between the parts and does not specify the function of each part

The hierarchy shows a *taxonomy* of the system parts, but *not* the procedure for decomposing the system into parts — how do we do it?

But first, **why** do we want to decompose systems?

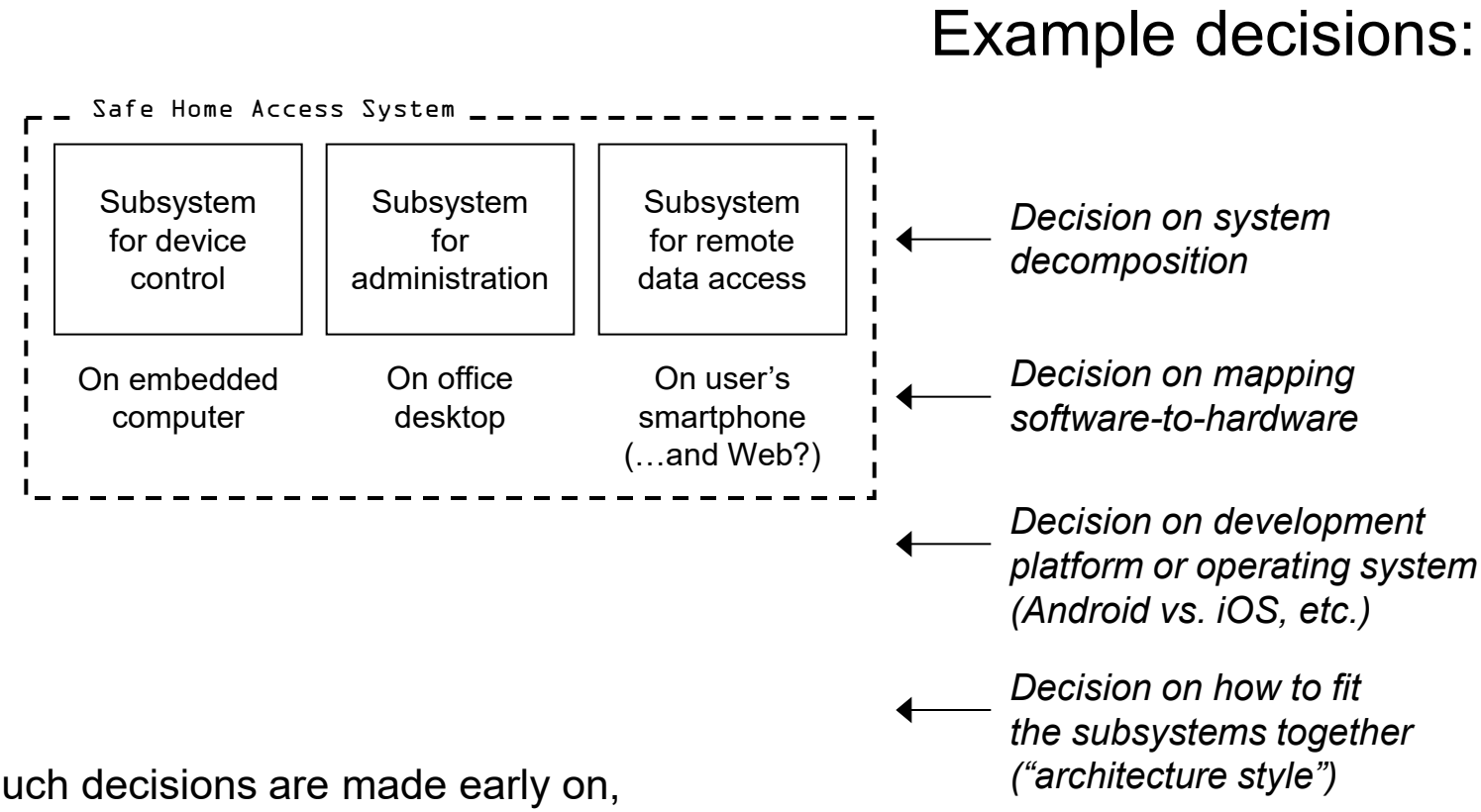
Why We Want To Decompose Systems

- Tackle complexity by “divide-and-conquer”
- See if some parts already exist & can be reused
- Focus on creative parts and avoid “reinventing the wheel”
- Support flexibility and future evolution by decoupling unrelated parts, so each can evolve separately (“separation of concerns”)

Software Architecture Definition

- Software Architecture =
  a set of high-level **decisions** that determine the structure of the solution
  (parts of system-to-be and their relationships)
  - Principal decisions made throughout the development *and* evolution of a software system
  - made early and affect large parts of the system ("design philosophy") — such decisions are hard to modify later
- Decisions to use well-known solutions that are proven to work for similar problems
- Software Architecture is **not** a phase of development
  - Does **not** refer to a **specific product** of a particular phase of the development process (labeled "high-level design" or "product design")

Example Architectural Decisions

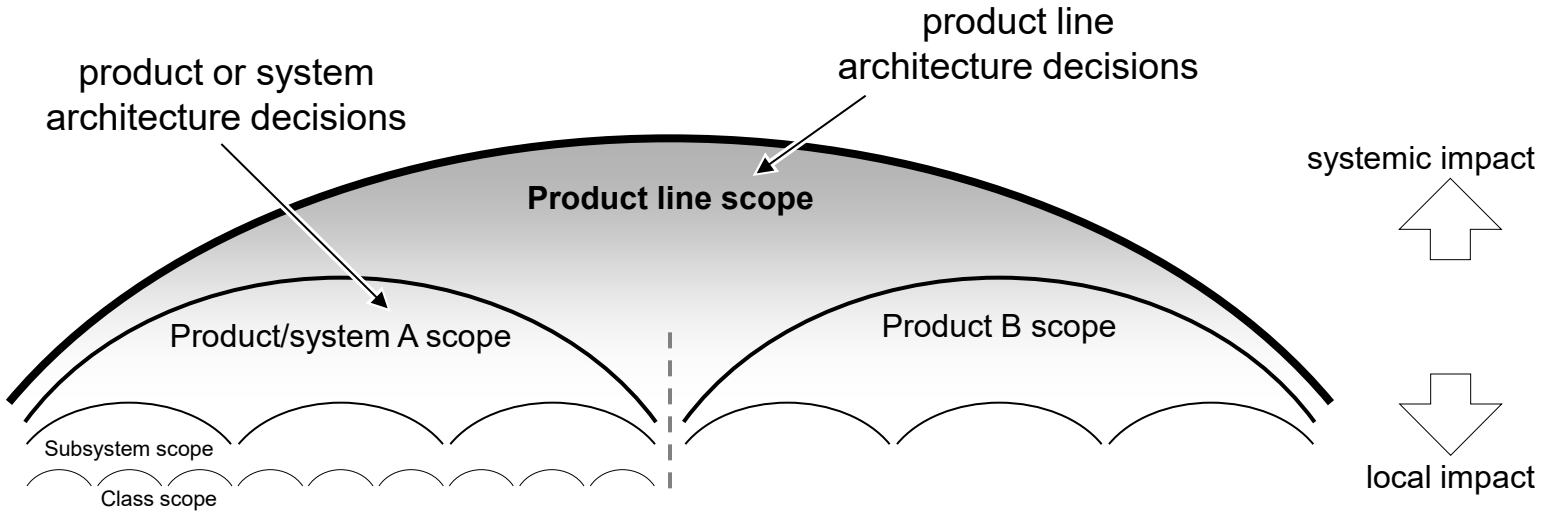


Such decisions are made early on,

perhaps while discussing the requirements with the customer

to decide which hardware devices will be used for user interaction and device control

Architectural Decisions
—A matter of scope



- Given the current level of system scope, a decision is "**architectural**" if it can be made only by considering the present scope
  - I.e. could not be made from a more narrowly-scoped, local perspective
  - Architectural decisions should focus on high impact, high priority areas that are in strong alignment with the business strategy

Software Architecture Key Concerns

(Principal decisions to be made)

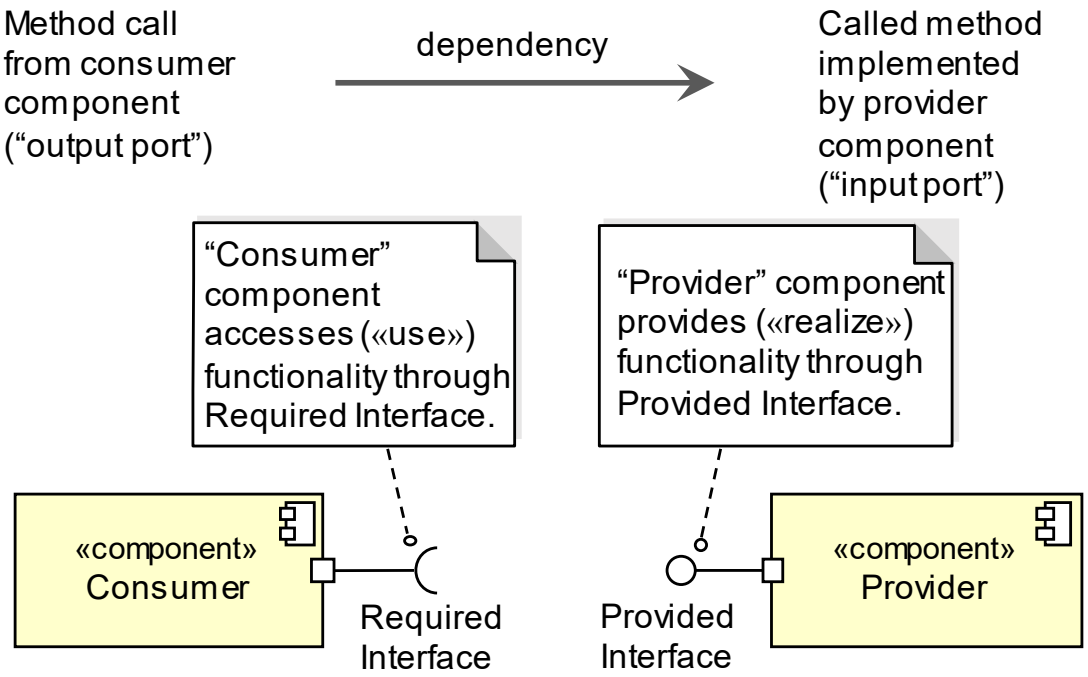
- System decomposition
  - how do we break the system up into pieces?
    - what functionality/processing or behavior/control to include?
  - do we have all the necessary pieces?
  - do the pieces *fit* together?
    - how the pieces interact with each other and with the runtime environment
- Cross-cutting concerns
  - broad-scoped qualities or properties of the system
  - tradeoffs among the qualities
- Conceptual integrity
- Software architecture provides a set of well-known solutions that are proven to work for similar problems

Architectural Decisions Often Involve Compromise

- The "best" design for a component considered in isolation may not be chosen when components considered together or within a broader context
  - Depends on what criteria are used to decide the "goodness" of a design
  - E.g., car components may be "best" for racing cars or "best" for luxury cars, but will not be best together
- Additional considerations include business priorities, available resources, core competences, target customers, competitors' moves, technology trends, existing investments, backward compatibility, ...

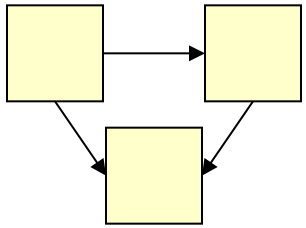
UML Notation for Software Components

- A component has its behavior defined in terms of **provided interfaces** and **required interfaces** (potentially exposed via ports)

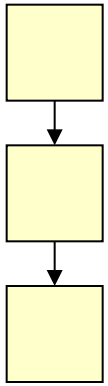


Module/Subsystem Views

- Module Decomposition View
  - Top-down refinement (e.g., simple “block diagram”)
- Dependency View
  - How parts relate to one another



- Layered View
  - Special case of dependency view



- Class View
  - “domain model” in OOA and “class diagram” in OOD

Component and Connector Views

- Process View
  - Defined sequence of activities?
    System represented as a series of communicating processes
- Concurrency View

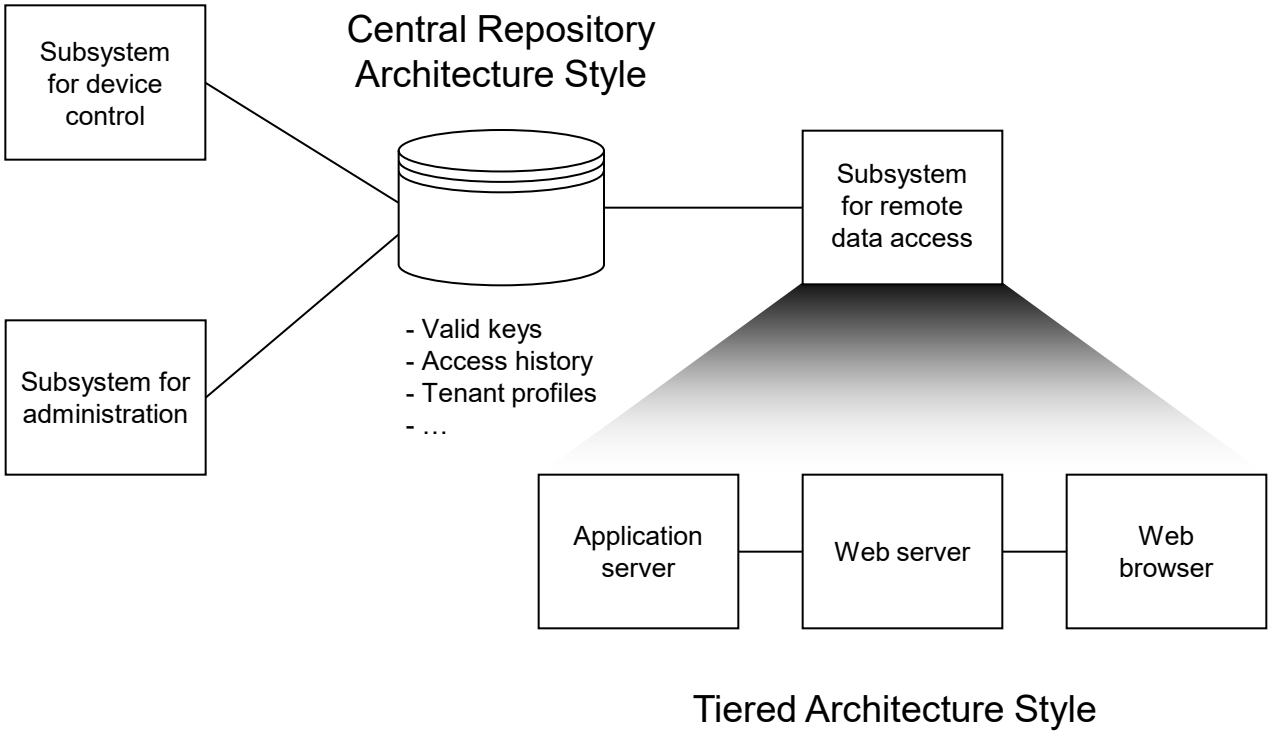
  …
- Client/Server View
  - E.g., in Web browsing

How to Fit Subsystems Together: Some Well-Known Architecture Styles

- World Wide Web architecture style: REST (Representational State Transfer)
- UNIX shell script architecture style: Pipe-and-Filter
- Client/Server
- Central Repository (database)
- Layered (or Multi-Tiered)
- Peer-to-Peer
- Microservices

Development platform (e.g., Web vs. mobile app, etc.) may dictate the architecture style or vice versa…

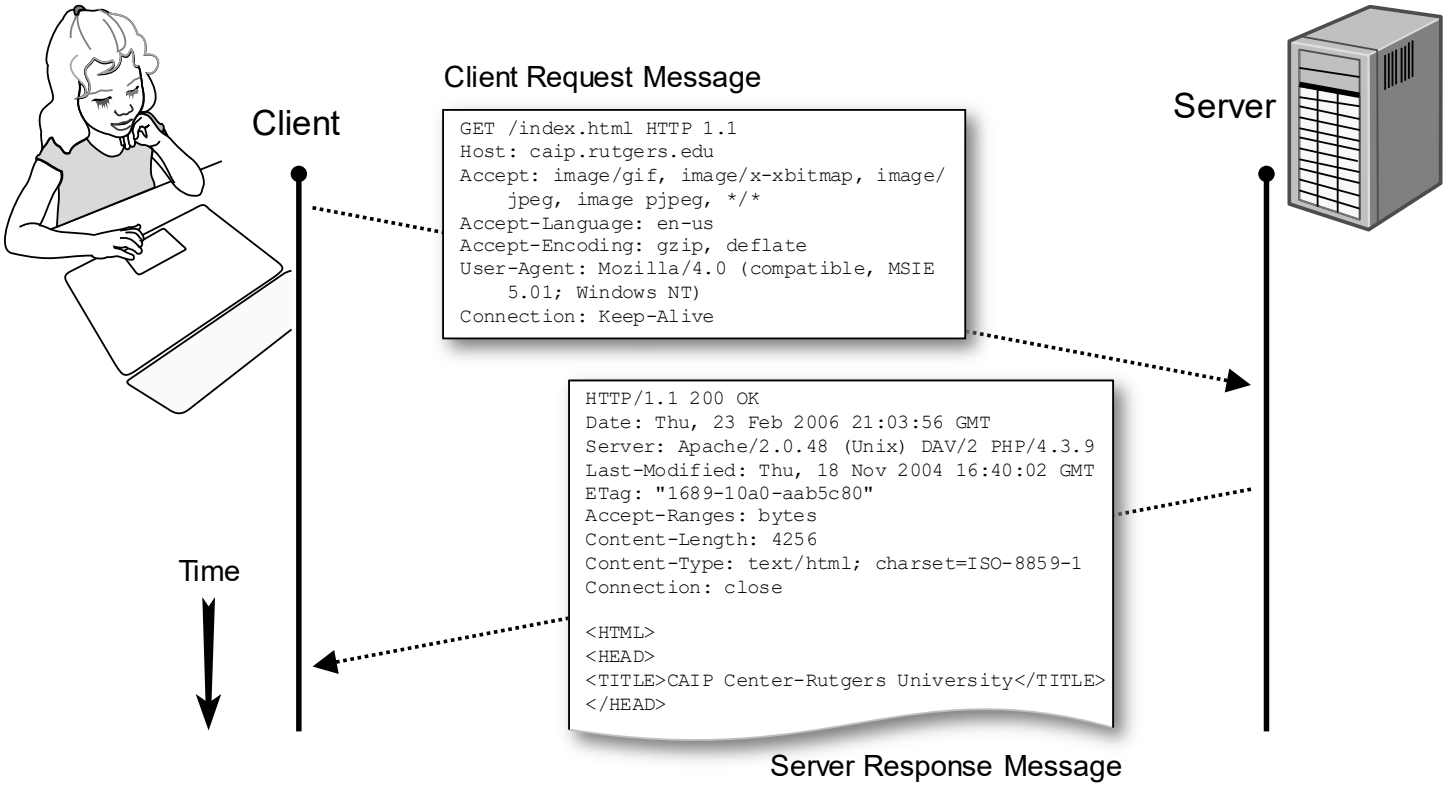
Real System is a Combination of Styles



Architecture Styles – Constituent Parts

1. Components
   - Processing elements that “do the work”
2. Connectors
   - Enable communication among components
     - Broadcast Bus, Middleware-enabled, implicit (events), explicit (procedure calls, explicit communications bus) …
3. Interfaces
   - Connection points on components and connectors
     - define where data may flow in and out of the components/connectors
4. Configurations
   - Arrangements of components and connectors that form an architecture

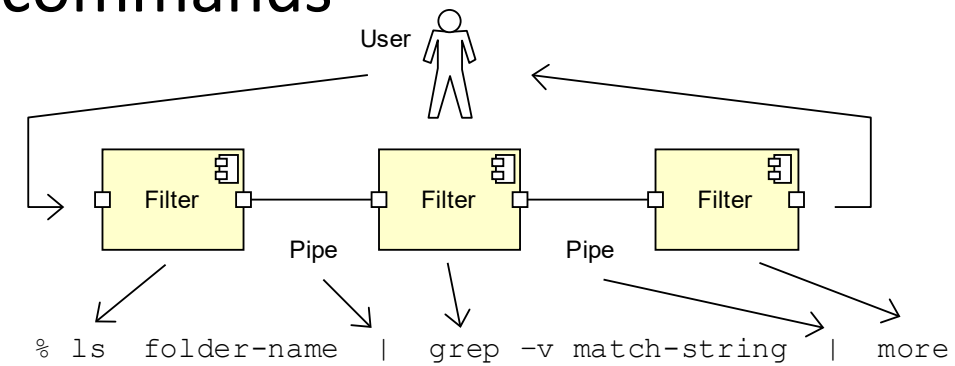
Connectors: HTTP Protocol



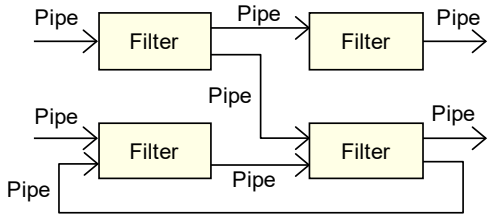
- POST method requests the server to **create** a new resource
- GET method requests the server to **read** a resource
- PUT method requests the server to **update** a resource
- DELETE method requests the server to **delete** a resource

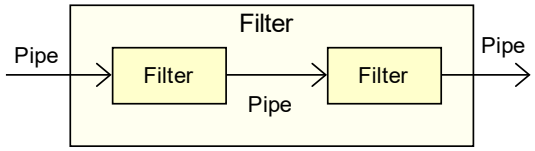
Architecture Style: Pipe-and-Filter

- Components: **Filters** transform input into output
- Connectors: **Pipe** data streams
- Example: UNIX shell commands



- More complex configurations:

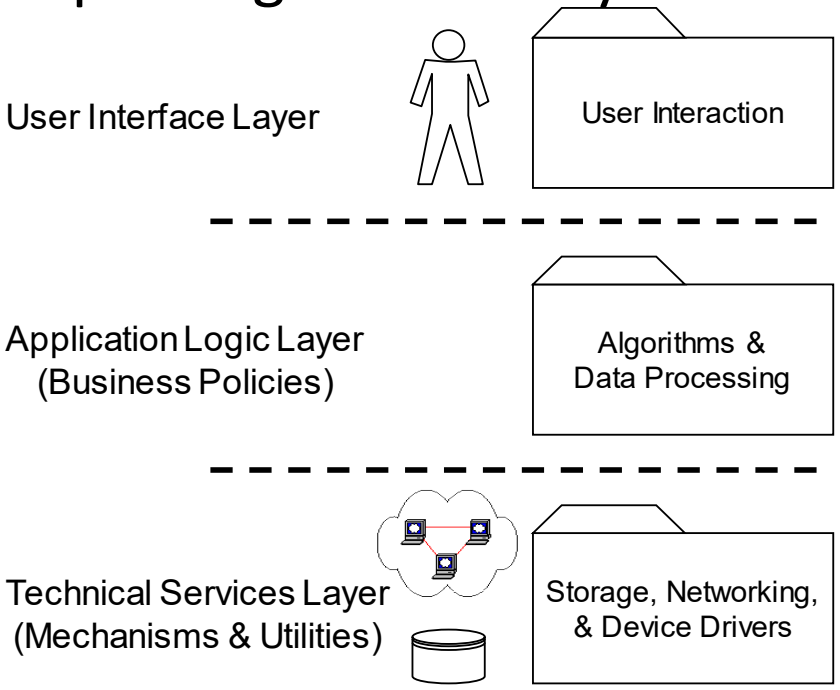




Architecture Style: Layered

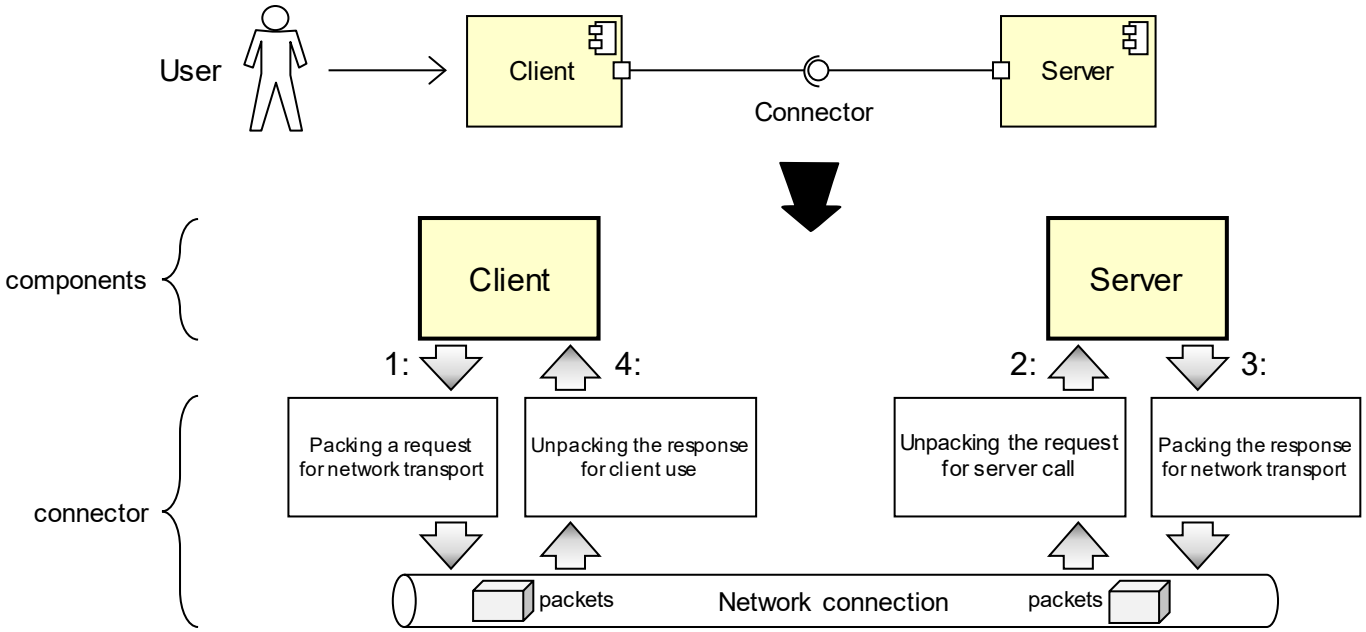
a.k.a. Tiered Software Architecture

- A layered system is organized hierarchically, each layer providing services to the layer above it and using services of the layer below it
- Layered systems reduce coupling across multiple layers by hiding the inner layers from all except the adjacent outer layer, thus improving evolvability and reusability

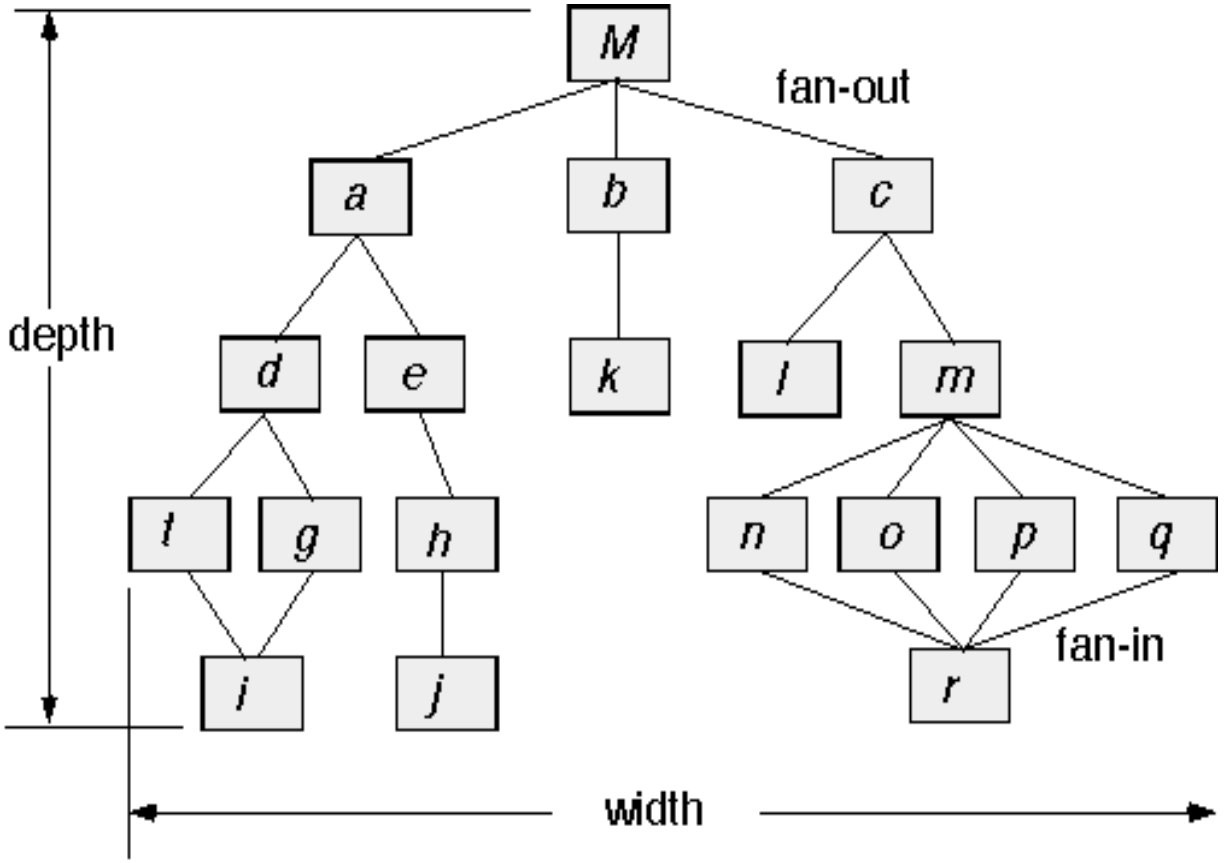


Architecture Style: Client/Server

- A **client** is a triggering process; a **server** is a reactive process. Clients make requests that trigger reactions from servers.
- A **server** component, offering a set of services, listens for requests upon those services. A server waits for requests to be made and then reacts to them.
- A **client** component, desiring that a service be performed, sends a request at times of its choosing to the server via a connector.
- The server either rejects or performs the request and sends a response back to the client

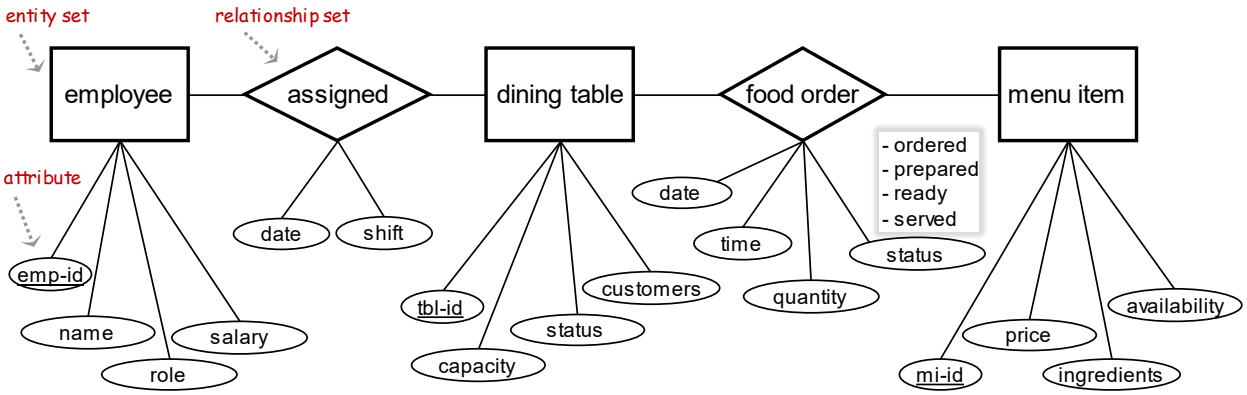


Call and Return Architecture

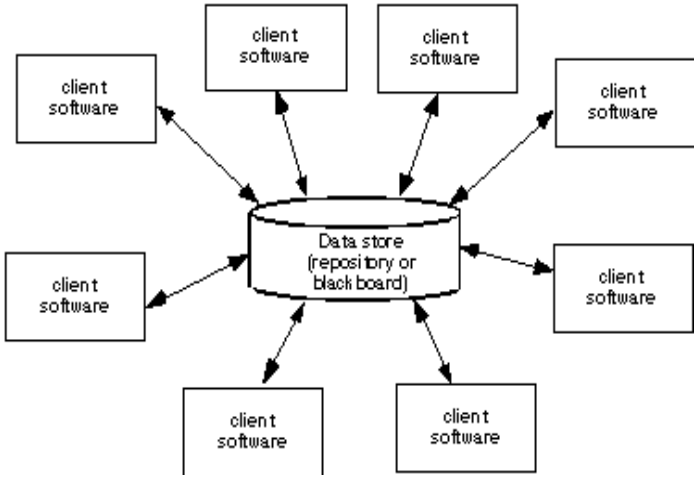


Shared Database Connector
— Data Model for Restaurant

Entity-relationship diagram of the data model:



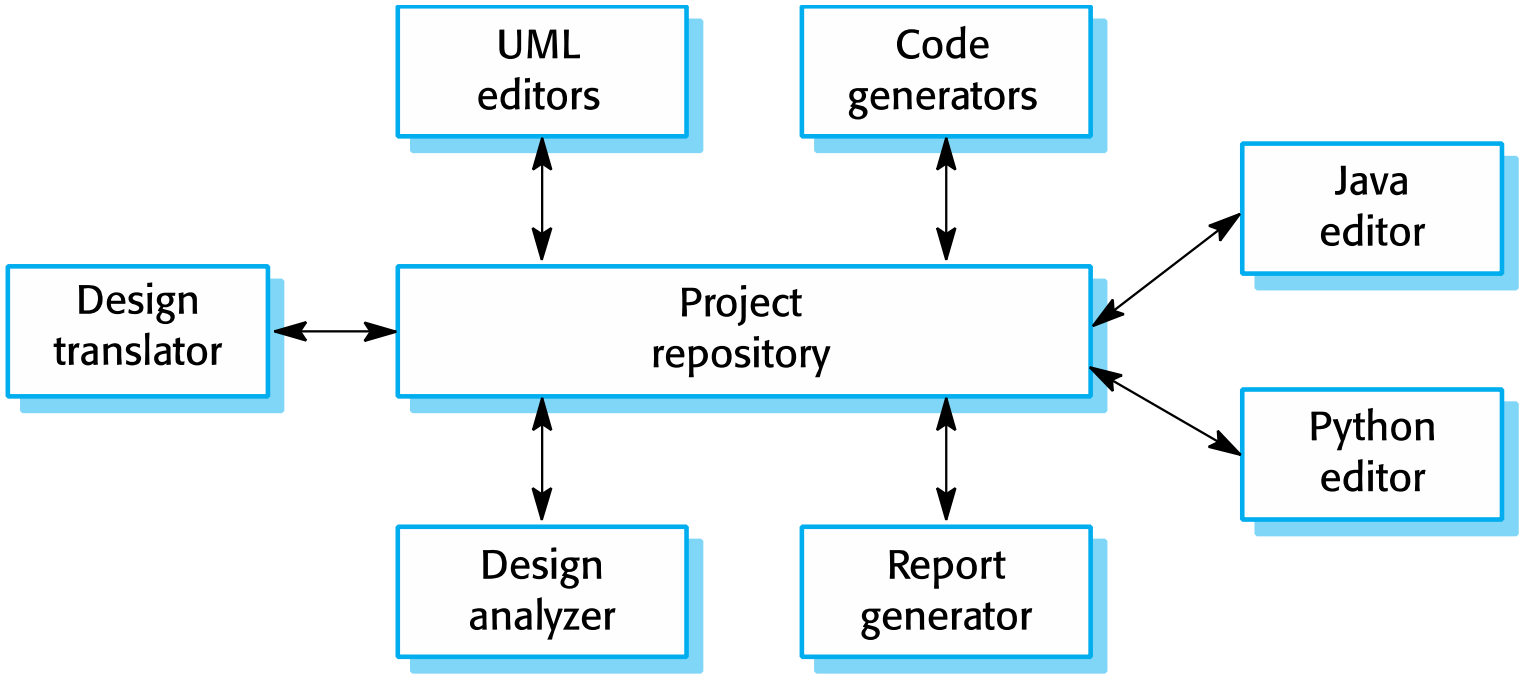
employee (identifier, name, role, salary, …)
assigned (date, working shift)
dining table (identifier, capacity, status, seated customers)
food order (time, item, quantity, total price, status)
menu item (identifier, price, ingredients, availability)
payment (time, date, amount, confirmation number)



Repository architecture

- Sub-systems must **exchange data**. This may be done in two ways:
  - Shared data is held in a central database or repository and may be accessed by all sub-systems;
  - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used and this is an **efficient data sharing mechanism**.

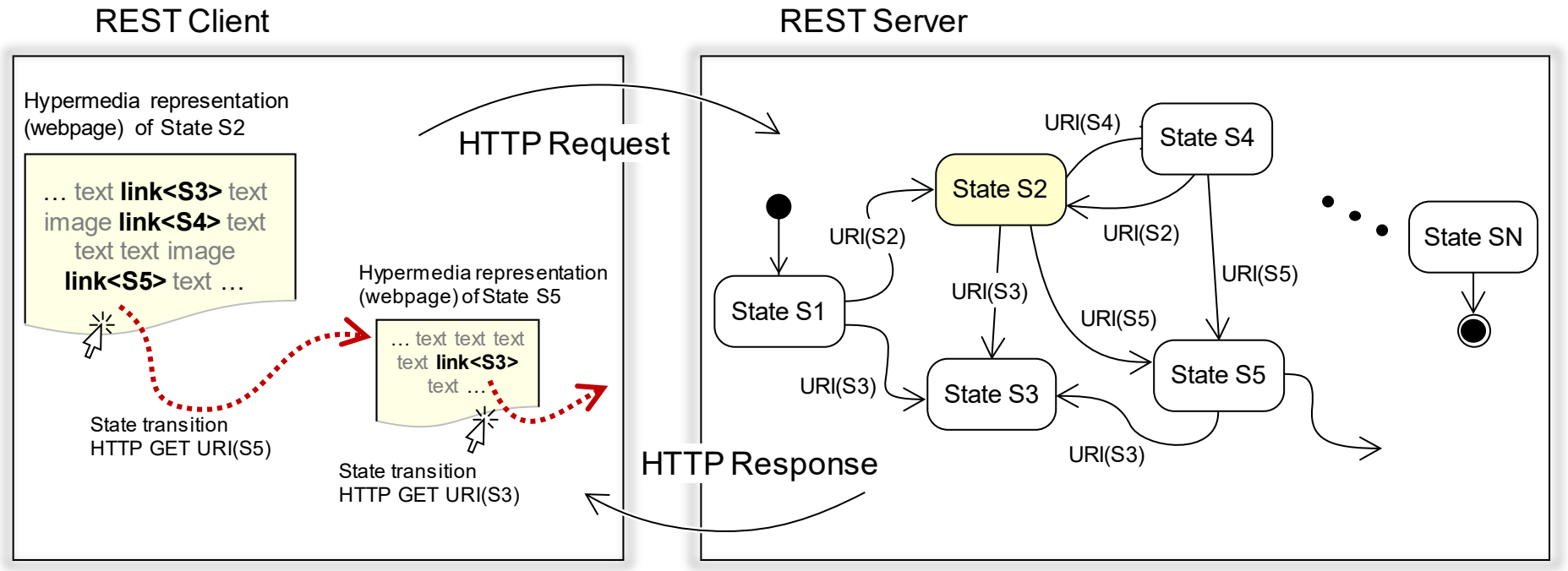
A repository architecture for an IDE



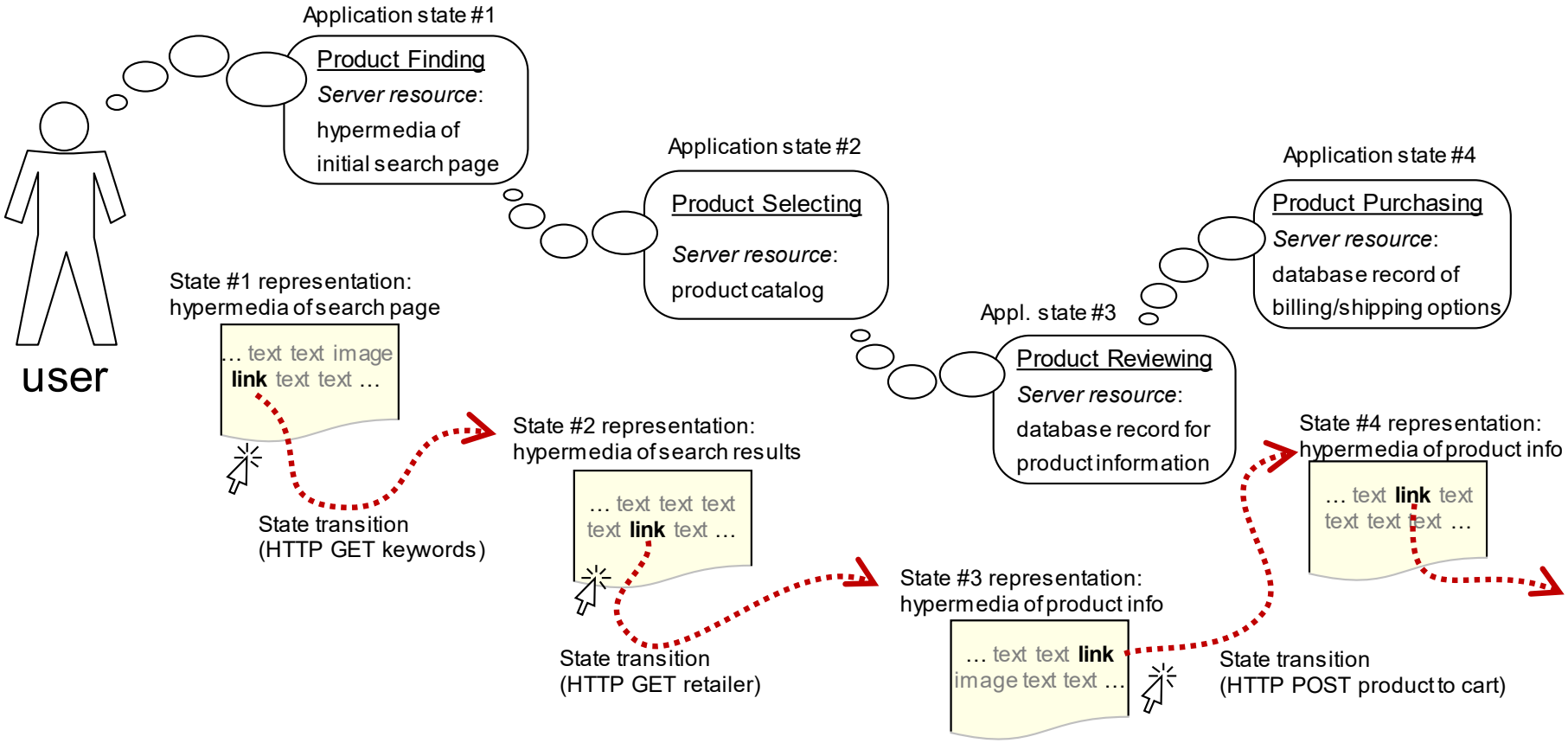
REST: Hypermedia as the Engine of Application State (HATEOAS)

REST: Representational state transfer

Conceptual model: hypermedia as the engine of application state (HATEOAS)

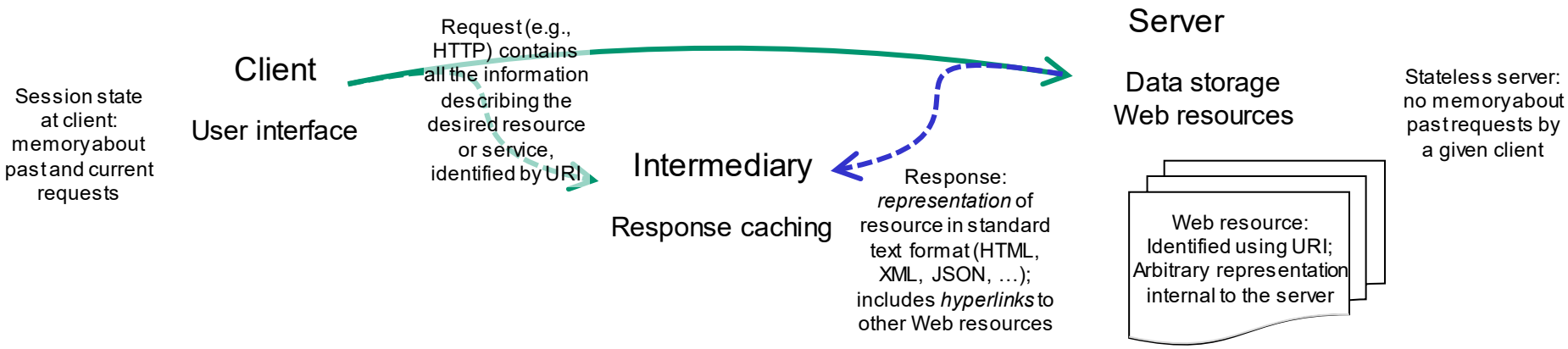


Hypermedia as the Engine of Application State



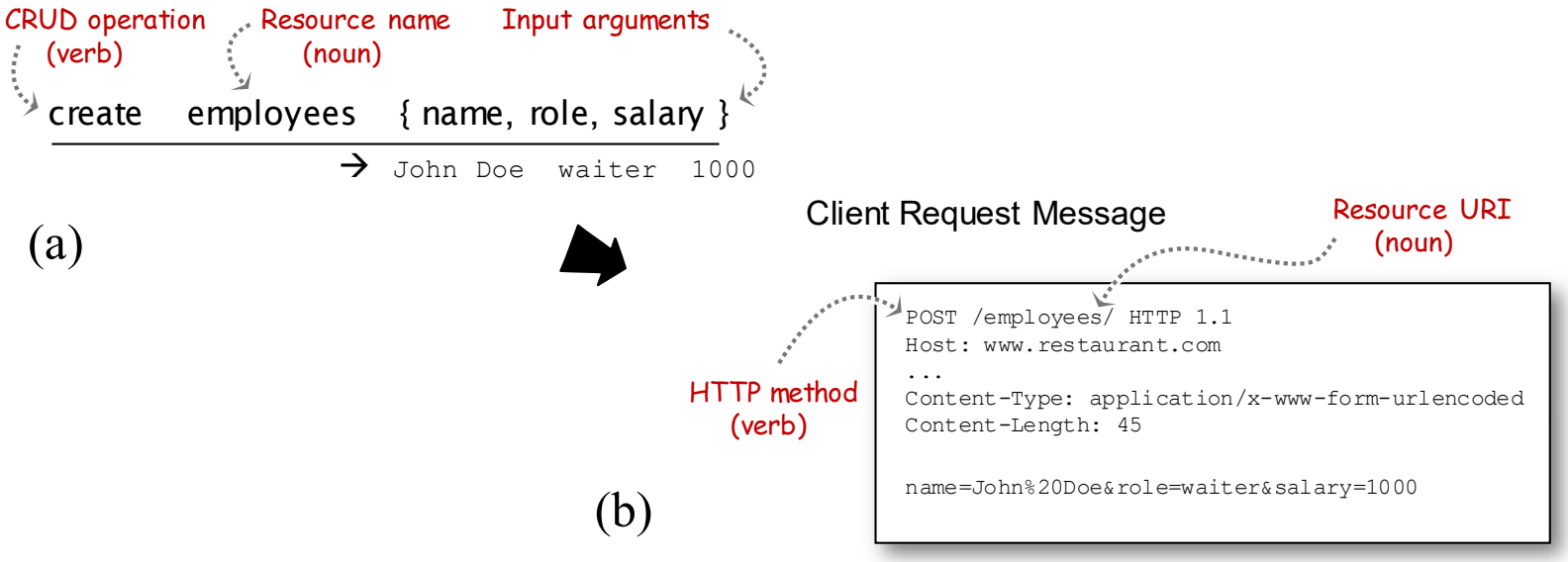
Web application:
A network of Web resources (a virtual state-machine), where the user by following the links requests the transfer of next resource (“state”).

REST physical model



RESTful Resource Naming

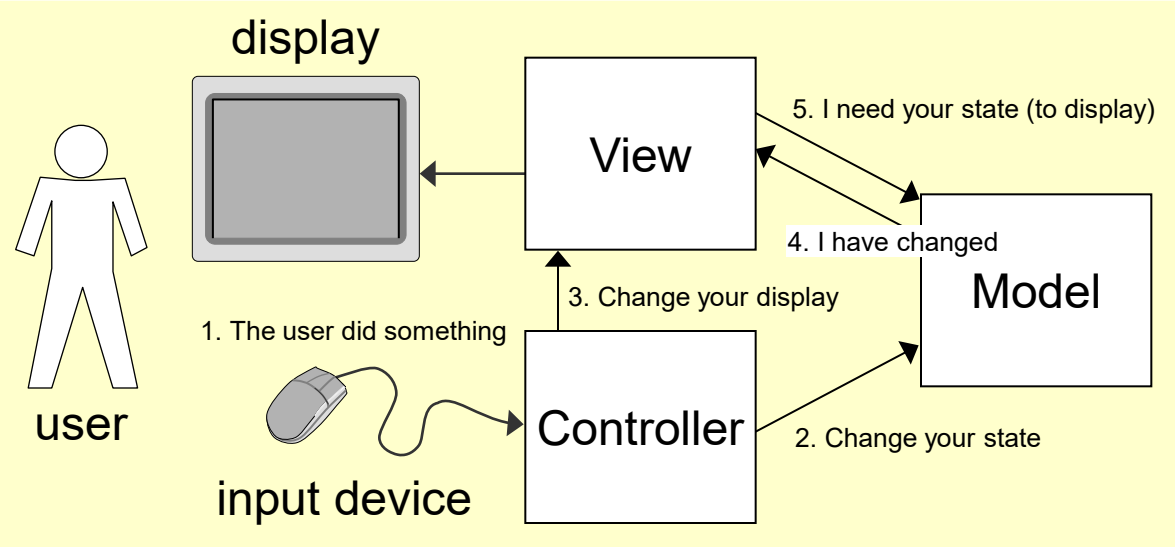
(a) Conceptual representation of creating a new employee record



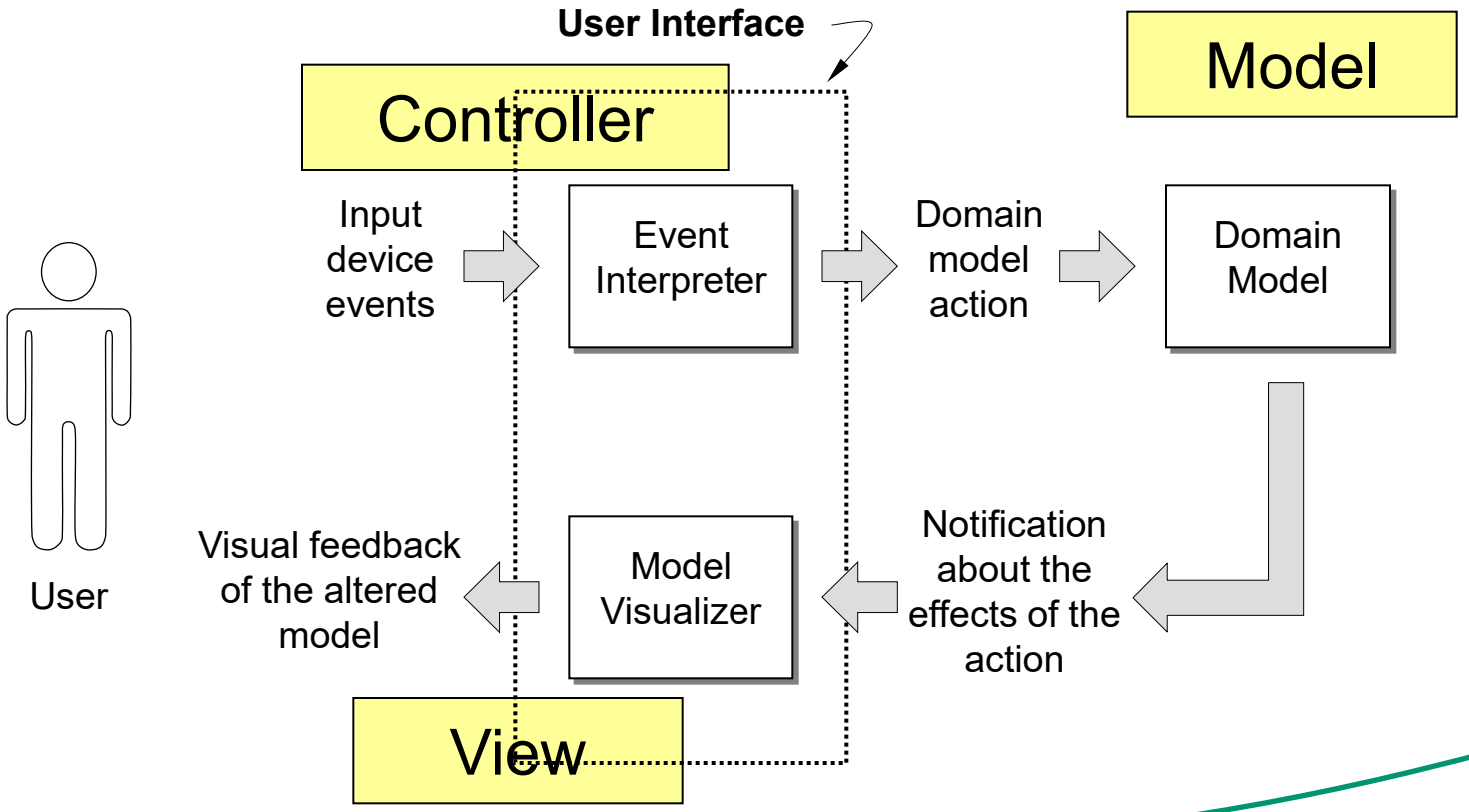
(b) HTTP request message carrying the content of the HTML form that was used to enter the employee data

Architecture Style: Model-View-Controller

- **Model**: holds all the data, state and application logic. Oblivious to the View and Controller. Provides API to retrieve state and send notifications of state changes to "observer"
- **View**: gives user a presentation of the Model. Gets data directly from the Model
- **Controller**: Takes user input and figures out what it means to the Model

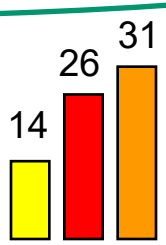


Model-View-Controller

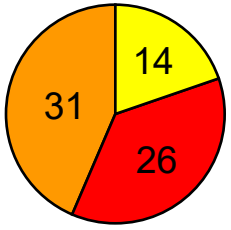


Model: array of numbers [14, 26, 31]

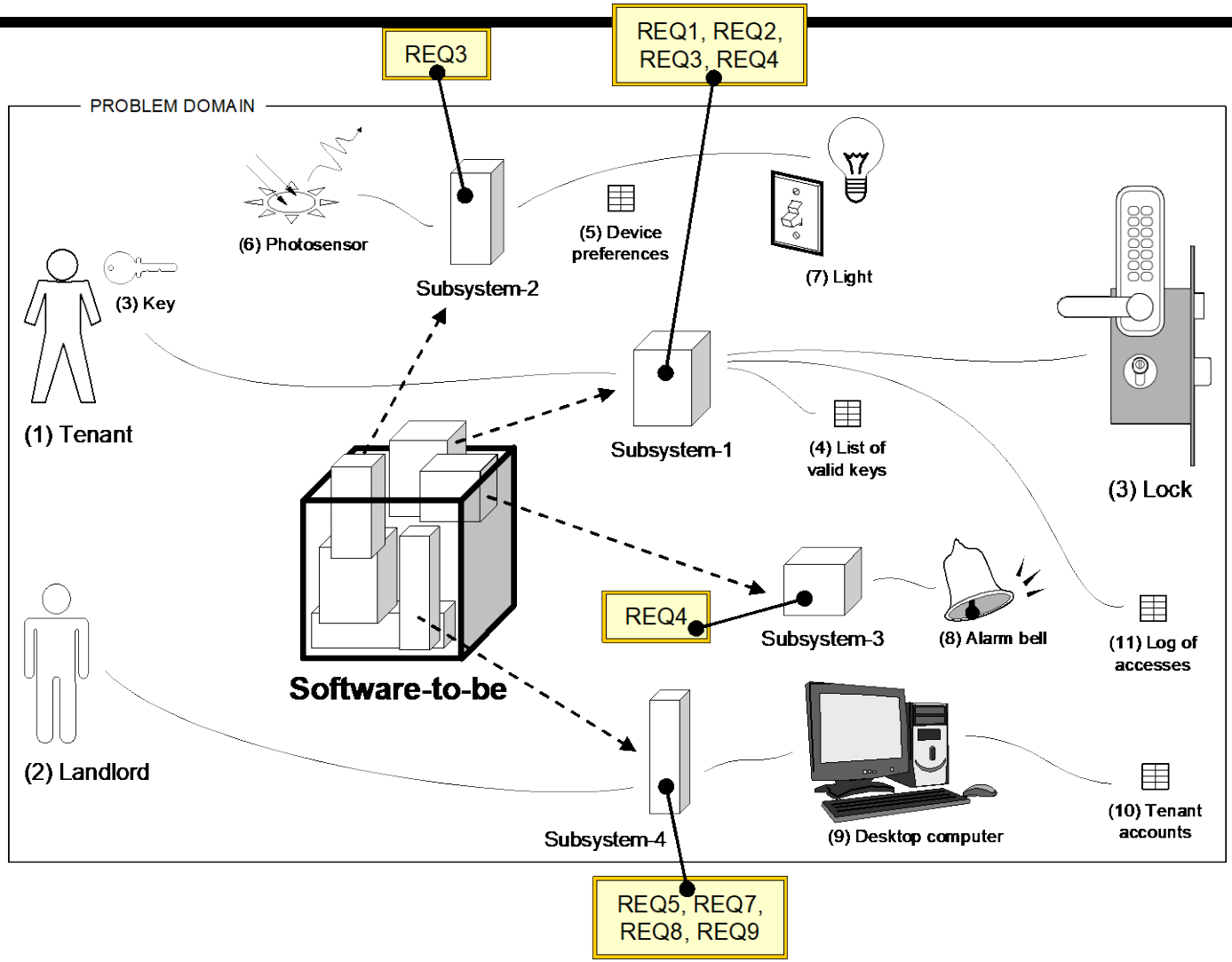
➔ Different Views for the same Model:



versus



Problem Structure

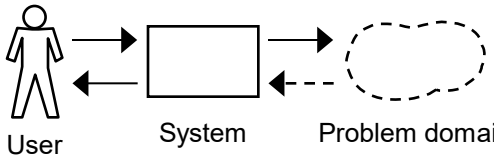


Subsystems derived from the requirements (“bottom-up” approach or induction)

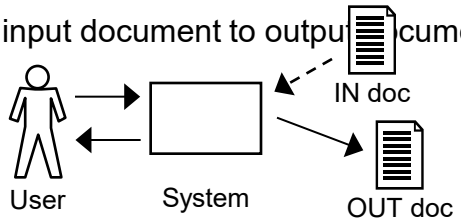
Typical Software Eng. Problems

1. User works with computer system (problem domain is “virtual”, not physical)

REQ-1: Map input data to output data as said by given rules

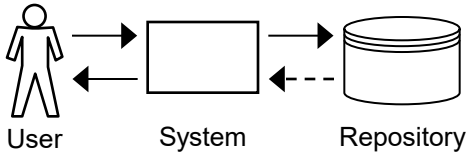


1.a) System transforms input document to output document



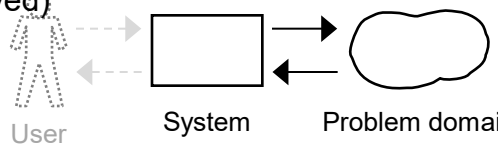
1.b) User edits information stored in a repository

REQ-2: Allow repository editing, where “repository” is a collection of data

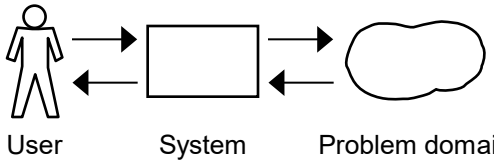


2. Computer system controls the (physical) problem domain (user not involved)

REQ-3: Autonomously control a physical object/device

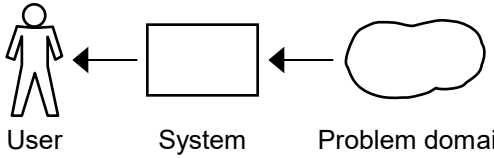


3. Computer system intermediates between the user and the problem domain



3.a) System observes the problem domain and displays information

REQ-5: Monitor and display information about an object



3.b) System controls the problem domain as commanded by the user

REQ-4: Interactively control a physical object/device

